Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP

Jose L. Muñoz, Oscar Esparza, Juanjo Alins, Jorge Mata
Network Engineering
Universitat Politècnica de Catalunya (UPC)

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outline

1 Sessions

2 SDP

3 SIP
- Introduction
- SIP basics
- Headers
- SIP Architecture
- DDDS & ENUM

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Multimedia Communications

- Information about media contents needs to be shared between principals, and sometimes with the network.
- Multimedia contents are mostly transmitted using sessions.
- A multimedia session is a set of multimedia senders, receivers and data streams flowing among them.
- A multimedia conference is an example of a multimedia session.
- Protocols are needed to:
  - Describe sessions (SDP)
  - Create/maintain/end sessions (SIP).
  - Announce multimedia services (SAP), control streaming (RTSP), or transmitting multimedia info (RTP/RTCP), etc.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outline

1 Sessions

2 SDP

3 SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SDP: Session Description Protocol

- IETF RFC 4556 (MMusic Group)
- SDP is not a typical protocol (with messages, headers,etc.).
- SDP is a format to describe all characteristics of a session in a text-based manner.
- SDP provides a general description of sessions, including:
  - Media details.
  - Transport protocols, addresses and other properties.
  - User information, session metadata, etc.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SDP Coding

SDP Coding

v=0	*Version Identifier*
o=g.company88772388 IN IP 4 152.246.133.22	*Session Identifier*
u=http://www.seefish.com	*URL*
e=test@company.com	*eMail*
c=IN IP 4 152.246.133.22	*Session-IP-Address*
b=CT:64	*Bandwidth (64 kbit/s)*
t=30862727236 0	*Start and Stop Time*
k=clear:manhole cover	*Key for Decoding*
m=audio 3456 RTP/AVP 96	*Media Description (Audio), Portnumber and protocol*
a=rtpmap:96 VDVI/8000/1	*Properties (RTP, Codec-Typ 96)*
m=video 3458 RTP/AVP 31	*Media Description (Audio), Portnumber and protocol*
m=application 32416 udp wb	*Media Description (Whiteboard), Portnumber and protocol*
a=orient:portait	*Properties (Arrangement)*

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SDP Usage Examples

- Session Announcement Protocol (SAP): an application-layer session directory protocol to communicate the relevant session information to conference participants
- Real Time Streaming Protocol (RTSP): an application-level protocol for control over the delivery of multimedia contents.
- Session Initiation Protocol (SIP): an application-layer control protocol for creating, modifying, and terminating sessions.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SDP purpose

- Describe all session properties, but specially, information about media streams.
- This will allow the participants of a session to properly understand the session.
- A SDP session description (must or may) includes:
  - Session name and purpose.
  - Contact information for the person responsible for the session
  - Time(s) the session is active (start and stop times bounding the session)
  - The type of media (video, audio, etc.)
  - The transport protocol (RTP, TCP, SCTP, etc.)
  - The format of the media (H.261 video, MPEG video, etc.)
  - Addresses (unicast or multicast) and ports.
  - Information about the bandwidth to be used by the session, etc.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Session Description Grammar

- A SDP session description consists of a number of lines of text in the form:
  `<type>=<value>`
- In the session description there is:
  - One session-level section:
    - It includes general information about the session.
    - It starts with a "`v=`" line
  - One or many media-level sections:
    - Each one includes information about each particular media.
    - Each one starts with an "`m=`" line.
- Some information is required and some optional (marked with *).
- Information must appear in exactly the order given here (for error detection purposes).

Session description elements

- v= (protocol version)
- o= (originator and session identifier)
- s= (session name)
- i=* (session information)
- u=* (URI of description)
- e=* (email address)
- p=* (phone number)
- c=* (connection information)
- b=* (zero or more bandwidth information lines)
- z=* (time zone adjustments)
- k=* (encryption key)
- a=* (zero or more session attribute lines)
- t= (time the session is active)
- r=* (zero or more repeat times)

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Media description elements

- m= (media name and transport address)
- i=* (media title)
- c=* (connection information)
- b=* (zero or more bandwidth information lines)
- k=* (encryption key)
- a=* (zero or more media attribute lines)

SDP example I

- v=0
- o=<username> <sess-id> <sess-version> <nettype> <addrtype> <unicast-address>
  `o=joe 2890844526 2890842807 IN IP4 10.47.16.5`
- s=<session name>
  `s=SDP Lecture`
- i=<session description>
  `i=A Seminar on the session description protocol`
- u=<uri>
  `u=http://www.example.com/seminars/sdp`
- e=<email-address>
  `e=j.doe@example.com (Jane Doe)`
- p=<phone-number>
  `p=+34 999555555`
- c=<nettype> <addrtype> <connection-address>[/<ttl>]
  `c=IN IP4 224.2.17.12/127`
- t=<start-time> <stop-time>
  `t=2873397496 2873404696`
- a=<attribute>
  `a=sendonly`

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SDP example II

- m=<media> <port> <proto> <fmt> ...
  <media> is the media type: audio, video, text, application, message, ...
  <port> is the transport port to which the media stream is sent.
  <proto> is the transport protocol: UDP, RTP/AVP, RTP/SAVP, ..
  <fmt> is a media format description: depends on the value of the <proto> (payload type numbers)

```
m=audio 49170 RTP/AVP 0
m=video 51372 RTP/AVP 99
a=rtpmap:99 h263-1998/90000
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SDP Drawbacks

- Limited expressiveness
- Very basic media descriptions available
- No real negotiation functionality
- There is a SDPng protocol with extended capabilities.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outline

1 Sessions

2 SDP

3 SIP
- Introduction
- SIP basics
- Headers
- SIP Architecture
- DDDS & ENUM

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outline

1 Sessions

2 SDP

3 SIP
- Introduction
- SIP basics
- Headers
- SIP Architecture
- DDDS & ENUM

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Introduction I

SIP (Session Initiation Protocol)

SIP is a signaling protocol widely used for the establishment, management and ending of multimedia sessions. Classical uses of multimedia sessions are voice calls, video conferencing, instant messaging or on-line gaming.

- SIP has its roots in the IP community rather than in the field of communications.
- It was defined by the IETF in RFC 3261.
- It comes from the combination of two protocols:
  - SIPv1: protocol designed for video conferencing, which includes the location of users.
  - SCIP: protocol for establishing multimedia sessions based on HTTP. SCIP used the e-mail address of users to identify them.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Introduction II

- SIP advantages over other protocols:
  - It is a general purpose protocol (not limited to IP telephony).
  - Simple text-based signaling.
  - SIP participants form an overlay
  - It allows to include external (non-standard) information:
    - It can include information about presence of users.
    - It uses standard protocols to define sessions (SDP).
  - It allows mobility: it maintains a centralized periodic record, generated by users who inform about their location.
  - It delegates much of the support services to users, unlike traditional telephony protocols as SS7.

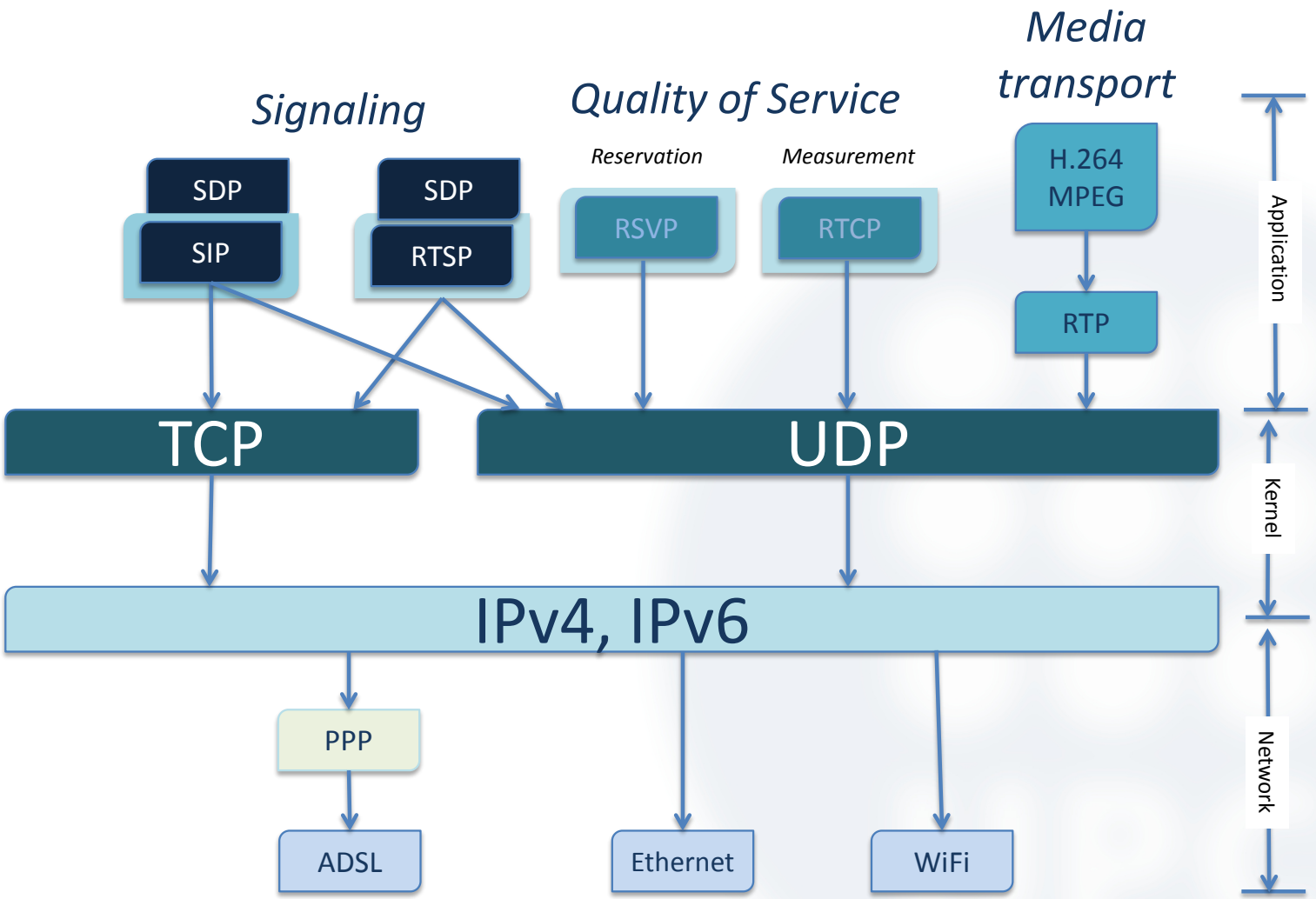
Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Introduction III

- Main functions:
  - User location: (geographical) location of the UA.
  - User availability: it distributes the user's status: available, busy or DND (Do not Disturb).
  - User capabilities: it allows sending the capabilities of the UA to generate sessions.
  - Session Configuration: it allows the establishment of SIP sessions.
  - Session Management: it allows the UA to terminate, to transfer or to make changes of the parameters of the session.
- SIP is the standard signaling protocol with wider acceptance for VoIP:
  - Replaces H.323 (ITU-T).
  - In November 2000 it was selected by the 3GPP group as signaling protocol for IMS (IP Multimedia Subsystem).

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

IETF Multimedia Protocols



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outline

1 Sessions

2 SDP

3 SIP
- Introduction
- SIP basics
- Headers
- SIP Architecture
- DDDS & ENUM

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP messages I

- Messages and header fields are very similar to HTTP.
- They are based on text and use UTF-8 encoding (ISO 10646).
- The format of SIP messages was defined in RFC 2822.
- **A SIP message is composed of:**

- Start-Line.
- One or more header fields (Message Headers)
- Empty line (CR + LF) indicating the end of the headers.
- Optionally, a Message Body.



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP messages II

- SIP messages can be of two types:
  - **Request** (methods).
  - **Responses** (response codes).
- The first line of a message determines whether it is a SIP request or a SIP response.
- In the requests:
  - The first line is called **request line**.
  - This **request line** contains the name of a method.
- In the responses:
  - The first line of the response starts with SIP/2.0.
  - This first line also contains a response code.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Example: INVITE Request

```
INVITE sip:alice@203.0.113.2 SIP/2.0
Via: SIP/2.0/TCP
198.51.100.3:54274;rport;branch=z9hG4bKPjImxzxCydmO63BYTMt3Qt6yEm1HL5IBZw;alias
Max-Forwards: 70
From: sip:bob@example.com;tag=vJtmQaD5mHLCIBlAeOdf55d.Mm.bOmXV
To: sip:alice@203.0.113.2
Contact: <sip:bob@198.51.100.3:5060;ob>
Call-ID: xB4IsFv1I4fX296-c2kio3UkgYQTX3Uo
CSeq: 30914 INVITE
Route: <sip:198.51.100.2;lr>
Allow: PRACK, INVITE, ACK, BYE, CANCEL, UPDATE, INFO, SUBSCRIBE, NOTIFY, REFER,
MESSAGE, OPTIONS
Supported: replaces, 100rel, timer, norefersub
Session-Expires: 1800
Min-SE: 90
User-Agent: PJSUA v2.3 Linux-3.3.8/i686/glibc-2.11
Proxy-Authorization: Digest username="bob", realm="example.com",
nonce="VhJzQFYSchTZCJ2sUfgCJXMp+Pkie4zI", uri="sip:alice@203.0.113.2",
response="4afc65aef1798105590c2ae645f89489"
Content-Type: application/sdp
Content-Length:   473

v=0
o=- 3653038228 3653038228 IN IP4 198.51.100.3
....
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Example: Response 200

```
SIP/2.0 200 OK
Via: SIP/2.0/TCP
198.51.100.3:54274;rport=54274;received=198.51.100.3;
branch=z9hG4bKPjImxzxCydmO63BYTMt3Qt6yEm1HL5IBZw;alias
Record-Route: <sip:203.0.113.2;lr>
Record-Route: <sip:198.51.100.2;lr;r2=on>
Record-Route: <sip:198.51.100.2;transport=tcp;lr;r2=on>
Call-ID: xB4IsFv1I4fX296-c2kio3UkgYQTX3Uo
From: <sip:bob@example.com>;tag=vJtmQaD5mHLCIBlAeOdf55d.Mm.bOmXV
To: <sip:alice@203.0.113.2>;tag=wyZIvK16DO8A04LtHQVXVZt3ahy46WK5
CSeq: 30914 INVITE
Contact: <sip:alice@203.0.113.3:5060;ob>
Allow: PRACK, INVITE, ACK, BYE, CANCEL, UPDATE, INFO, SUBSCRIBE, NOTIFY, REFER,
MESSAGE, OPTIONS
Supported: replaces, 100rel, timer, norefersub
Session-Expires: 1800;refresher=uac
Require: timer
Content-Type: appli`` call '' cation/sdp
Content-Length:   272
...
```

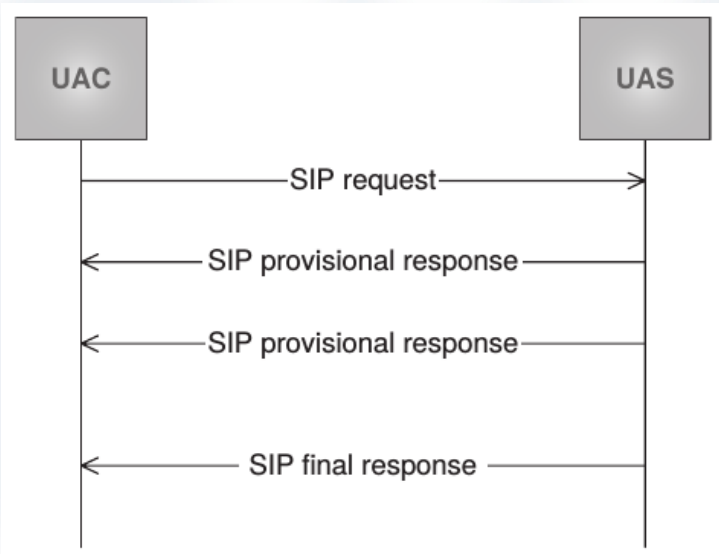
Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP Architecture

- SIP has two fundamental entities:
  - **User Agents (UA)**.
    - They are end entities able to manage SIP sessions.
    - **UAC (User agent client)**, which is a logical entity that creates and sends SIP requests.
    - **UAS (User Agent Server)**, which is a logical entity that generates responses to requests.
    - During a session, a UA usually operates as both a UAC and UAS.
    - In addition, the UA must be compatible with the Session Description Protocol (SDP), which provides the description of the multimedia session.
  - **SIP servers**.
    - They manage the flow of SIP messages and, among other functions, they help in contacting UAs.

Transactions and Dialogs I

- There are two different types of “conversations” between SIP elements: dialogues and transactions.
  - A SIP **transaction** is a request/response exchange between two adjacent SIP entities (at the same level).
  - A SIP **dialogue** includes all messages related to an end to end “ call ”.
- A transaction starts with a request and ends, in general, with a final response.
- The transaction may also contain provisional responses before the final one.



- As an exception, some methods may not have a final answer (basically, the ACK method).

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

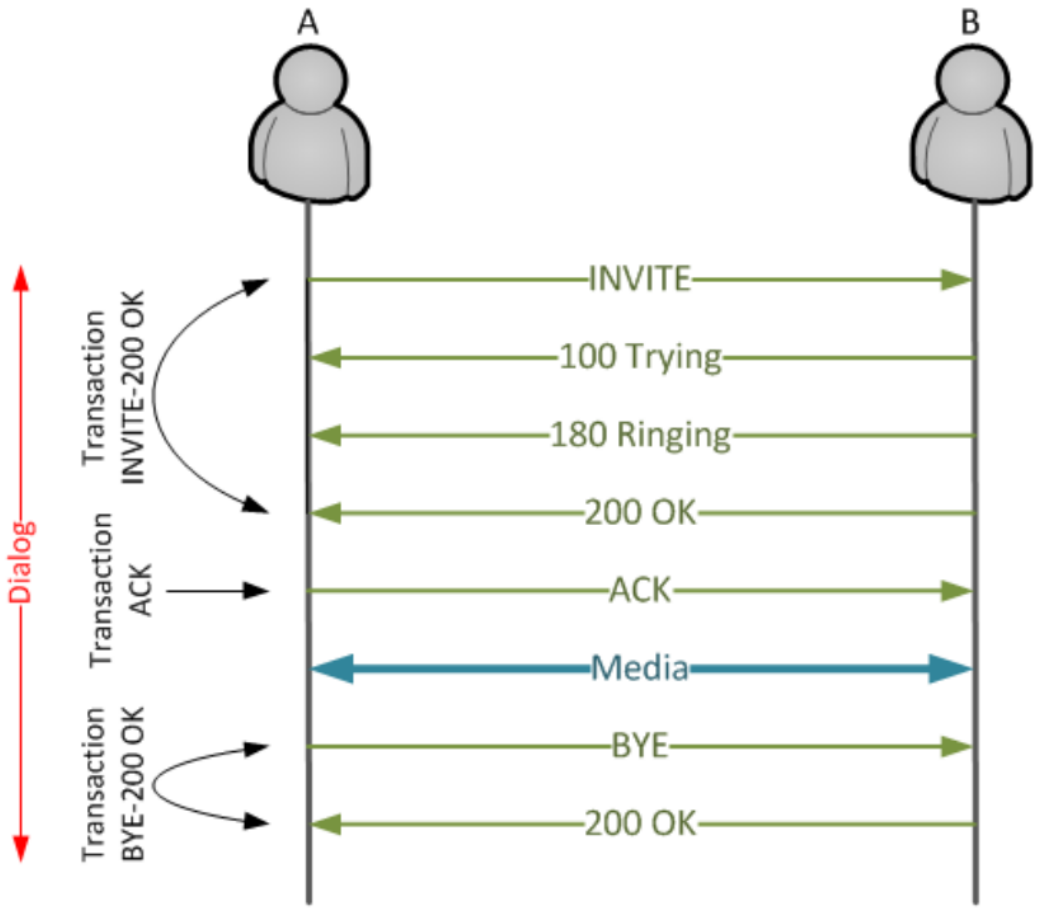
Transactions and Dialogs II

- **Identifiers:**
  - A transaction is identified unambiguously by the `branch` field of the **`Via`** header.
  - A dialogue is unambiguously identified by the fields `tag` in the **`From`** and **`To`** headers, and by the **`Call-ID`** header.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Transactions and Dialogs III

- Dialogue and transaction in a SIP session:



- Transactions are actually **parts of a dialogue**.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Transactions and Dialogs IV

- **INVITE Transaction**:
  - It starts with the INVITE request.
  - It ends with a 200 OK response code (in case of success call).
  - It may have intermediate responses 100 Trying and 180 Ringing.
- **ACK Transaction:**
  - In general, the ACK has its own transaction (not part of the INVITE transaction).
  - However, when the final response does not have a 2xx code, then the ACK is part of the INVITE transaction.
- Note that a *3-way handshake* (INVITE-OK-ACK) is used to start a session.
- Next, the media flow is sent between UAs.
- **BYE Transaction**:
  - To terminate a SIP session, a user ends the call by sending a BYE request.
  - The other user responds with a 200 OK response.

Request format

- SIP requests have an initial line (request-line), headers, and optionally a body.
- The request line is composed by:
  - A method name: INVITE, BYE, etc.
  - ID of the recipient of the request: request URI (which is a SIP URI).
  - Version of the SIP protocol.
  - Carriage return and line feed (CR + LF characters).
  - Example:

```
INVITE sip:bob@example.com SIP/2.0
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Methods

- SIP requests are also called methods.
- The initial SIP protocol (RFC3261) defined **six methods**:
  1. *INVITE*: used to establish a session between UAs.
  2. *ACK*: confirmation of a SIP session establishment.
  3. *CANCEL*: it ends a pending request (not an established session).
  4. *BYE*: it ends an established session.
  5. *REGISTER*: used by a UA to register in a SIP “Registrar“ server.
  6. *OPTIONS*: it asks information to another UA (without establishing a session). This information includes supported methods, content types, extensions, codecs, etc. The information is provided in a 200 OK response.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Extended methods

- In additional RFCs some methods were defined to offer additional functionalities.
- The extended main methods are:
  - *SUBSCRIBE*: subscription request to receive notifications concerning the events of a SIP user (presence).
  - *NOTIFY*: notifies an event to a subscriber.
  - *PUBLISH*: publishes an event on a server. The server will generate as many NOTIFY as subscribers there are.
  - *REFER*: indicates that the sender wants to be notified about the result of another request.
  - *INFO*: allows to send control information at the application level in an established session.
  - *UPDATE*: modifies the status of a multimedia session.
  - *MESSAGE*: used to send instant messages using SIP.
  - *PRACK*: ensures reliable and ordered delivery of provisional responses in SIP.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP URIs I

- SIP uses URIs (Uniform Resource Indicators) to route requests.
- URIs can also appear in various SIP headers and messages.
- Format of a SIP URI:

```
sip:user:password@host:port;uri-parameters?headers
```

- Example:

```
sip:alice@example.org:1234;user=phone?priority=urgent
```

- *user:password*: identifies a user (with password) in the domain of the request.
- *host*: identifies the host that provides a particular resource. It can be a domain name, an IPv4 address or an IPv6 address.
- *port*: identifies the port to which the request is sent.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP URIs II

- *uri-parameters*: additional parameters, separated by semicolons.
  - Examples: `transport`, `maddr`, `ttl`, `user`, `method` and `lr`.
  - The parameter `transport` specifies the transport protocol that it is used (UDP, TCP, SCTP, etc.).
  - The parameter `lr` (discussed later) is to route messages in a loose way.
- *headers*: used in case the URI is used to construct new requests.
  - The headers present in the SIP URI are included in the new request.
  - They are specified behind ”?”
  - They are encoded as hname=hvalue value pairs, separated by ampersands (&).

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Responses

- Very similar to HTTP responses.
- They consist of a numeric code and a description.
- They are divided into six categories according to the first digit of the answer code.

1xx	Provisional	Request received. Continuing to process the request.
2xx	Success	The action was successfully received, understood, and accepted.
3xx	Redirection	Further action needs to be taken in order to complete the request.
4xx	Client Error	The request contains bad syntax or cannot be fulfilled at this server.
5xx	Server Error	The server failed to fulfill an apparently valid request.
6xx	Global Failure	The request cannot be fulfilled at any server.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

1XX Responses

- 1XX: provisional and informative responses: they are sent before the establishment of a session.
  1XX usual response codes:
  - 100: Trying
  - 180: Ringing
  - 181: Call is Being Forwarded
  - 182: Queued
  - 183: Session in Progress

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

2XX and 3XX Responses

- 2XX: Success responses that indicate acceptance of the request method.
  2XX usual response codes:
  - 200: OK
  - 202: Accepted (used with the REFER method).

- 3XX Redirection responses.
  3XX usual response codes:
  - 300: Multiple Choices
  - 301: Moved Permanently
  - 302: Moved Temporarily
  - 305: Use Proxy
  - 380: Alternative Service

4XX Responses

- 4XX: Responses indicating that the session cannot be established because of an error in the client side.
  4XX usual response codes:

- 400: Bad request.
- 401: Unauthorized. Used only in the registration process.
- 402: Payment Required.
- 403: Forbidden. Lack of permissions, for example to call international destinations.
- 404: Not Found: User not found, the terminal is not registered.
- 405: Method Not Allowed
- 406: Not Acceptable: when there are compatibility problems in the media.
- 407: Proxy authentication required.
- 408: Request Timeout.
- 480: Temporarily Unavailable.
- 481: Call/Transaction Does Not Exist.
- 482: Loop detected.
- 483: Too many hops.
- 484: Address Incomplete.
- 486: Busy Here.
- 487: Request Terminated.
- 491: Request Pending.
- 493: Undecipherable: could not decrypt the S/MIME body part.

5XX Responses

- 5XX: Responses that indicate some kind of error in a SIP server.
- Examples: no available data channels, network overload or not reachable numbering.
  5XX usual response codes:
  - 500: Server Internal Error
  - 501: Not implemented (the SIP request is not implemented)
  - 502: Bad Gateway
  - 503: Service Unavailable
  - 504: Server Time-out
  - 505: Version Not Supported (the server does not support this version of the SIP protocol)
  - 513: Message Too large

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

6XX Responses

- 6XX: Reponses that indicates some sort of overall failure.
- These codes are used when the application cannot be served in any server.
  6XX usual response codes:
  - 600: Busy everywhere
  - 603: Decline
  - 604: Does Not Exist Anywhere
  - 606: Not Acceptable

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outline

1 Sessions

2 SDP

3 SIP
- Introduction
- SIP basics
- Headers
- SIP Architecture
- DDDS & ENUM

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Headers

- They are also called **SIP fields**.
- They provide attributes to the SIP message.
- They are similar to the fields of the HTTP headers.
- They follow this format:

```
Header: Value1;Value2;...
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Required Headers

- The following headers are required in any SIP request:
  - **`Via`**
  - **`Max-Forwards`**
  - **`To`**
  - **`From`**
  - **`Call-ID`**
  - **`CSeq`**
- Some other headers may be optionally included by SIP specific requests.
- The header **`MaxForwards`** contains the maximum number of SIP hops allowed (it minimizes the damage of a loop).

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Headers `To`, `From` and `Call-ID` I

- The headers **`From`** and **`To`** are used to identify the sender and the receiver (respectively) of a request.
- When the receiver sends a response, **`From`** and **`To`** are not switched (they always indicate from where to where the request goes).
- The parameter `tag` in headers **`From`** and **`To`** together with the header **`Call-ID`** identify unambiguosly a SIP dialog.
- **`From`** and **`To`** are used to identify the ends of a dialogue, but **NOT to route messages**.
- How SIP messages are routed:
  - Requests are routed using the request-URI of the **`request line`**.
  - Responses are routed using the `Via` header.

Headers **To**, **From** and **Call-ID** II

- When a session (dialogue) is established, the issuer:
  - It randomly generates the `tag` parameter in header **From**.
  - It randomly generates a value for the **Call-ID**.

```
INVITE sip:alice@203.0.113.2 SIP/2.0
...
From: sip:bob@example.com;tag=vJtmQaD5mHLCIBlAeOdf55d.Mm.bOmXV
To: sip:alice@203.0.113.2
Call-ID: xB4IsFv1I4fX296-c2kio3UkgYQTX3Uo
```

- The receiver:
  - It randomly generates the `tag` parameter in the **To** header, and adds it to the parameters received from the issuer.

```
SIP/2.0 200 OK
...
From: <sip:bob@example.com>;tag=vJtmQaD5mHLCIBlAeOdf55d.Mm.bOmXV
To: <sip:alice@203.0.113.2>;tag=wyZIvK16DO8A04LtHQVXVZt3ahy46WK5
Call-ID: xB4IsFv1I4fX296-c2kio3UkgYQTX3Uo
....
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

`Via` Header I

- The **`Via`** header is the first to appear in SIP messages after the start line.
- There may appear several **`Via`** headers in a message.
- Each **`Via`** header contains:
  - SIP version and the transport protocol.
  - The URI of a SIP entity that forwarded that SIP message (previous hops).
  - A random string in the `branch` parameter.
- Examples of **`Via`** header:

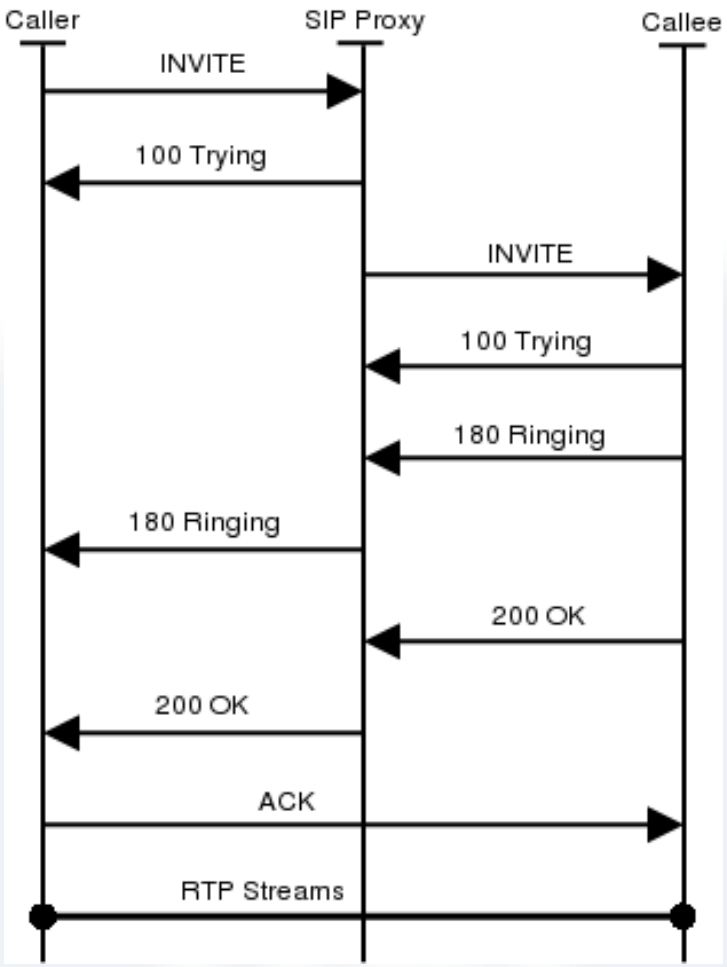
```
Via: SIP/2.0/UDP 200.20.20.2;branch=z9hG4bKb06c.0597.0
Via: SIP/2.0/UDP 10.20.20.4:5060;received=10.200.15.253;branch=z9hG523;
rport=5060
```

- The **`Via`** header is used to identify transactions, route responses, and also to avoid loops.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

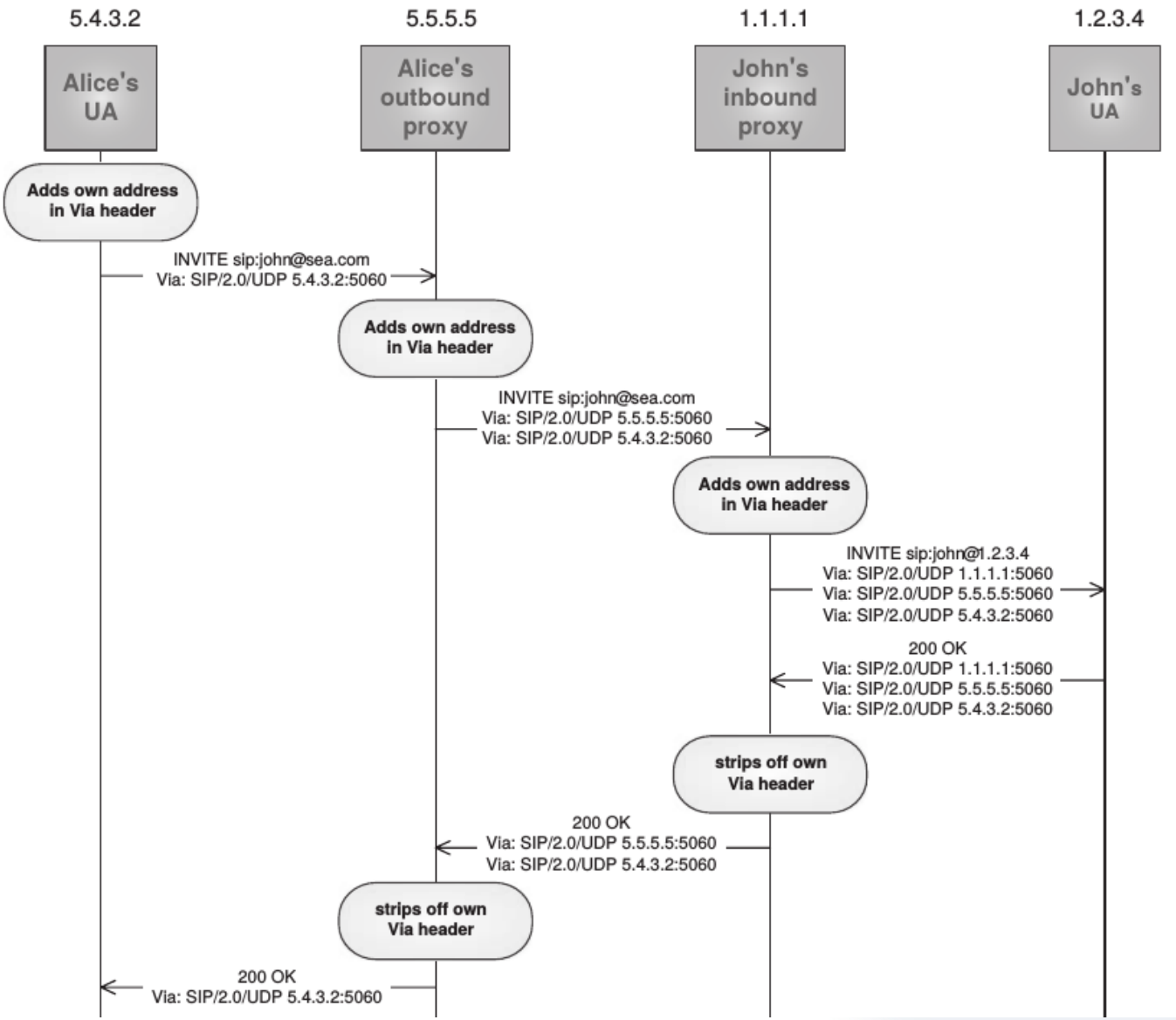
`Via` Header II

- The value of `branch`:
  - It is used as a transaction identifier.
  - It allows to correlate responses to a request (all have the same value `branch`).
  - Important: a transaction has **sense between two adjacent devices** at the same SIP level.
  - Responses that traverse different SIP devices generate **new transactions** (different `branch` values).
  - In the figure we have 3 transactions.



`Via` Header III

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

`Via` Header IV

- SIP routing:
  - Each SIP device that creates or forwards a SIP message includes its **`Via`** header.
  - Between SIP entities there can be any number of IP hops.
  - The **`Via`** headers make responses to follow the same path (at SIP level) than requests.
- The parameter `received`:
  - It is used when a server receives a request, and the IP source address in the IP layer does not match the address in the previous **`Via`** header.
  - Then, to properly route back the message, the server adds the `received` parameter, which includes the real IP address from which the request was sent.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

`Cseq` Header I

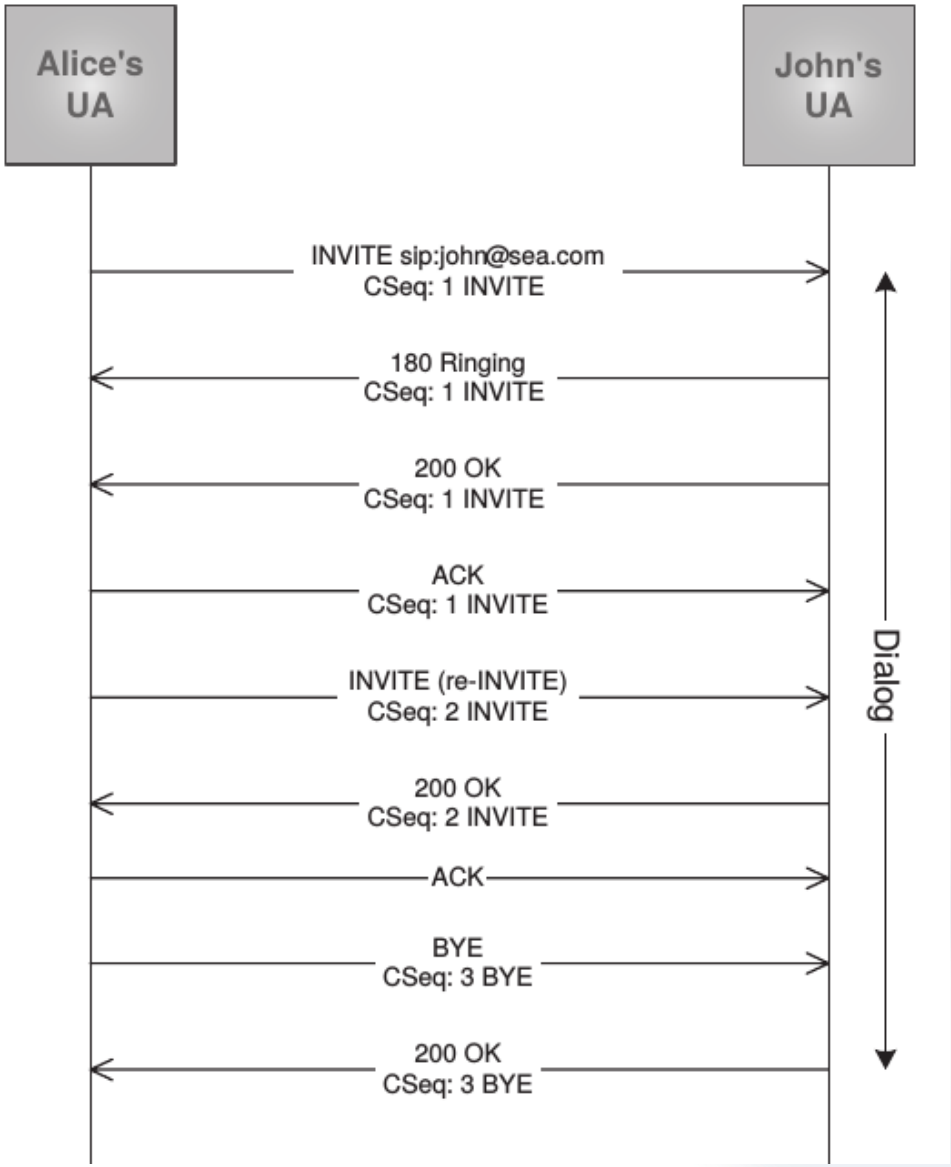
- **`Cseq`**: sequence number of the message.

```
Cseq: 3 INVITE
```

- The sequence number is used to order end to end requests within a dialog:
  - A UA should increase by 1 each new request it sends.
  - Thus, the receiver UA can determine whether a request has arrived out of order.
  - The exception is that the ACK and CANCEL requests do not cause an increase in the sequence number.
  - The reason is that these requests are not always end to end.
  - The ACK requests have the same number in **`CSeq`** than the INVITE they recognize.
  - The CANCEL requests have the same number in **`CSeq`** than the transaction they are canceling.
  - This is used to distinguish a response to a CANCEL to a response to an INVITE, the **`CSeq`** field will be different.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Cseq Header II

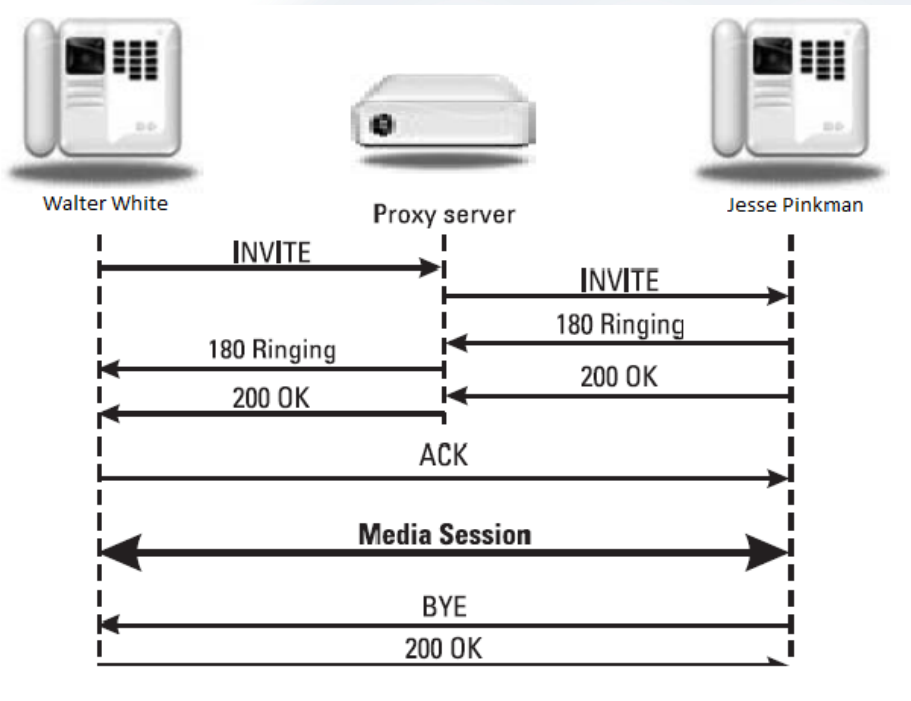


Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

`Contact` Header

- It indicates to a remote UA where to send future SIP requests within a dialogue.
- The **`Contact`** header contains a SIP URI.

- This is the default routing, and this allows to SIP proxies **not to be in the path of next requests** once UAs have contacted.
- Some service providers do not like this option as they cannot control traffic between entities (problems for billing).



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

`Record-Route` Header I

- As previously mentioned, by default proxies disappear from the SIP path in next requests.
- However, a proxy may want to stay in this signaling path, so future requests of the dialogue can be sent through this proxy.
- To do so, this proxy uses the **`Record-Route`** header, including its SIP URI.
- The **`Record-Route`** header is often used in the INVITE request.
- The UA that receives the INVITE request just copies the **`Record-Route`** headers received, so they can be received by the issuer.
- Notice that responses are sent back through the path ruled by the stack of **`Via`** headers.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

`Record-Route` Header II

- Hence, **`Record-Route`** headers are not necessary to determine the trajectory of responses.
- Unlike **`Via`** headers, **`Record-Route`** headers will not be removed when the UA response goes towards the origin.
- When the source UA receives the response of the first transaction, both ends will have the same stack of **`Record-Route`** headers (a list of intermediate points).

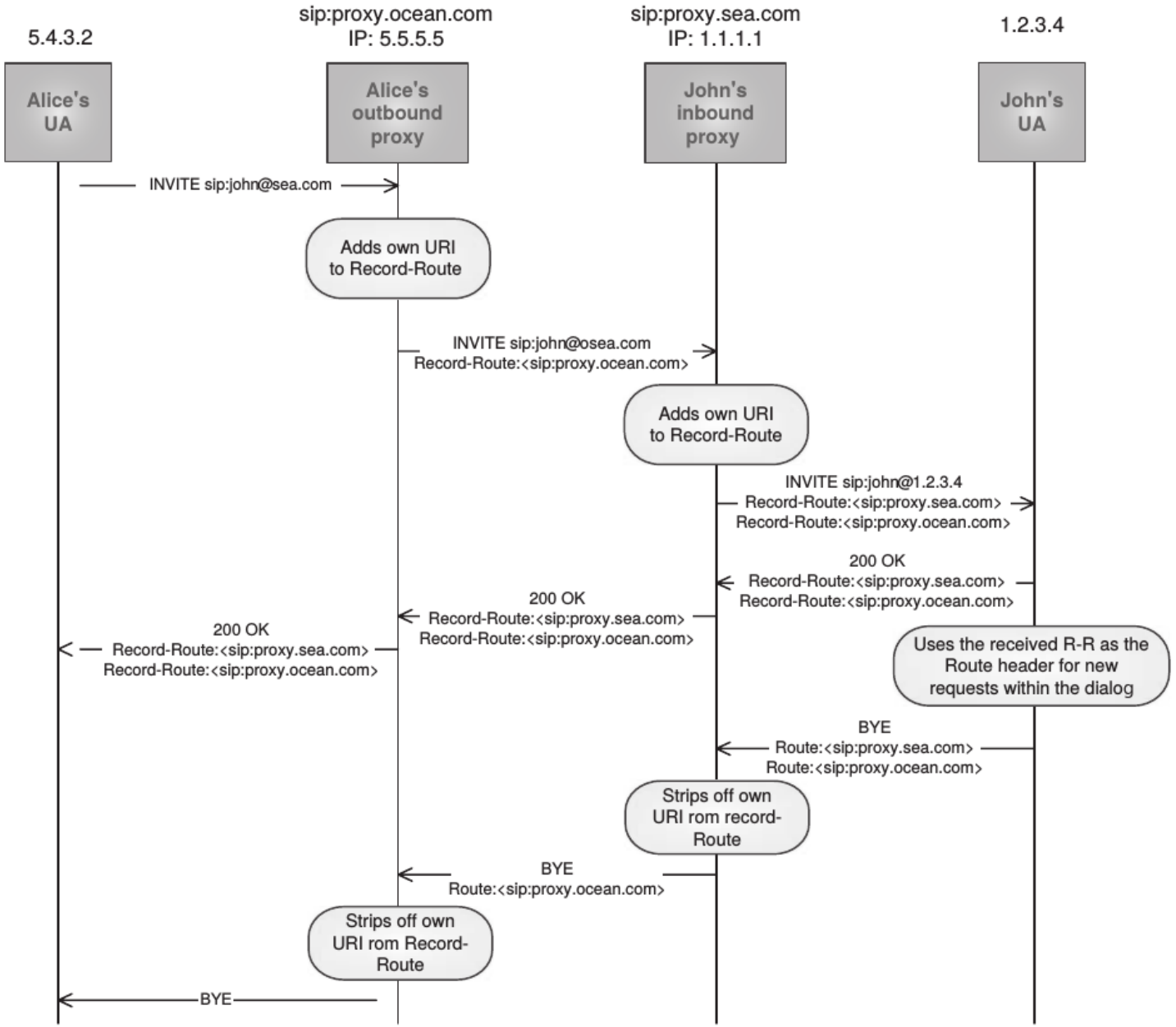
Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

`Route` Header I

- In general, a stack of **`Route`** headers forces to route the SIP requests through the provided list of proxies.
- The **`Record-Route`** and **`Route`** mechanisms work as follows:
  - When a UA must generate a new request, it includes the stack of routes learned, but changing **`Record-Route`** to **`Route`**.
  - To follow the same path, the destination UA will directly include the learned stack of routes.
  - On its side, the origin UA will include the learned stack of routes, but in reverse order.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Route Header II



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Types of SIP routing I

- **SIP strict routing**:
  - The Request-URI is rewritten with the value of the previous `Route` header, and then the header is deleted prior to send the message to this Request-URI.
  - The Request-URI contains the URI of the next hop.
  - The next hop may be another proxy or the destination UA.
  - As the Request-URI is rewritten, it is necessary to save the Original Request-URI in the last `Route` header.
  - Strict routing **is considered obsolete**.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Types of SIP routing II

- **SIP loose routing**:
  - The Request-URI field is not modified in the SIP hops.
  - The following SIP hop which at some point must pass the message is given by the previous **`Route`** header.
  - This is a significant change, now the Request-URI does not necessarily indicate where the request will be sent.
  - To indicate that a device supports SIP loose routing, the `lr` parameter is used:

```
Record-Route: <sip:10.31.101.50;lr>
Route: <sip:172.20.120.10;lr>
```

  - The last **`Route`** header will contain the data of the last proxy on the path.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Other Headers

- **`Content-Type`** and **`Content-Length`**: Description and size of the message body.

```
Content-Type: application/sdp
Content-Length: 151
```

- **`User-Agent`**: Description of terminal type that causes the message.

```
User-Agent: PJSUA v2.3 Linux-3.3.8/i686/glibc-2.11
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Message Body

- Optional.
- Usually, it contains SDP information that describes the multimedia session.
- Example of SDP information within an INVITE request:

```
v=0
o=- 3653038228 3653038228 IN IP4 198.51.100.3
s=pjmedia
b=AS:84
t=0 0
a=X-nat:0
m=audio 4000 RTP/AVP 98 97 99 104 3 0 8 9 96
c=IN IP4 198.51.100.3
b=TIAS:64000
a=rtcp:4001 IN IP4 198.51.100.3
a=sendrecv
a=rtpmap:98 speex/16000
a=rtpmap:8 PCMA/8000
a=fmtp:104 mode=30
a=rtpmap:96 telephone-event/8000
a=fmtp:96 0-16
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outline

1 Sessions

2 SDP

3 SIP
- Introduction
- SIP basics
- Headers
- SIP Architecture
- DDDS & ENUM

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP Entities I

1. **User Agents (UA)**:
   - They are end entities capable of managing SIP signaling and multimedia sessions (RTP).
   - The **Back-to-Back UA (B2BUA)** divides the communication channel into two call legs.
2. **SIP servers**:
   - They are network equipment to handle SIP signaling:
     - **SIP Proxy Server**: signaling routing stateful/stateless.
     - **SIP Redirect Server**: call redirection to another UA.
     - **SIP Registrar**: User registration and location information.
   - They are not involved in multimedia flows.

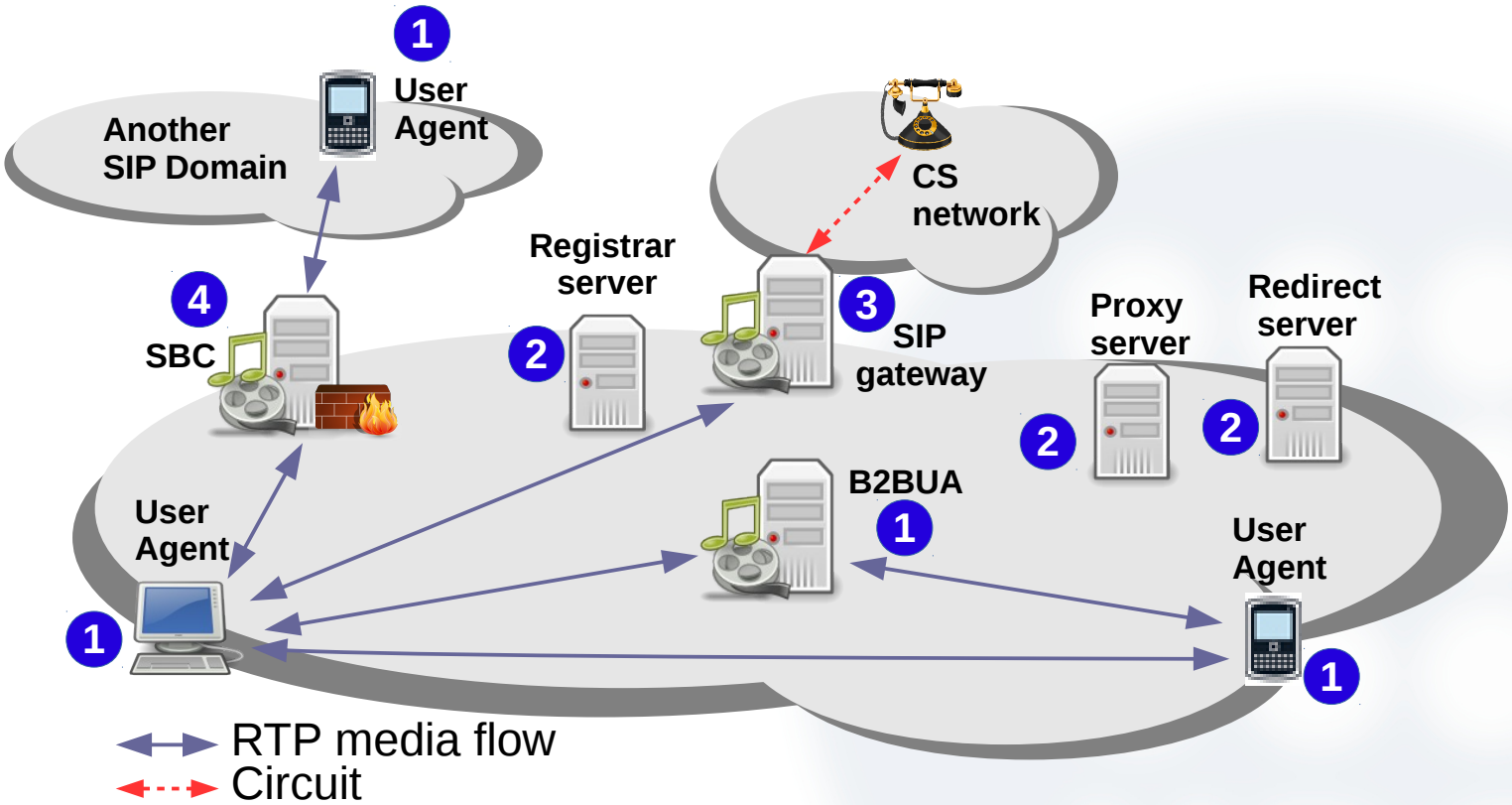
Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP Entities II

3. **Gateways**:
   - They allow the exchange of calls between circuit networks and SIP networks.
   - They manage translations between SIP and CS signaling (SS7).
   - They manage translations RTP-Circuits (transcoding).
4. **Session Border Controller (SBC)**:
   - They perform functions of firewalling.
   - They are usually placed between SIP domains (although sometimes they are also installed internally).
   - If you want to control the media streams they are usually configured as B2BUA.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP Entities III



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

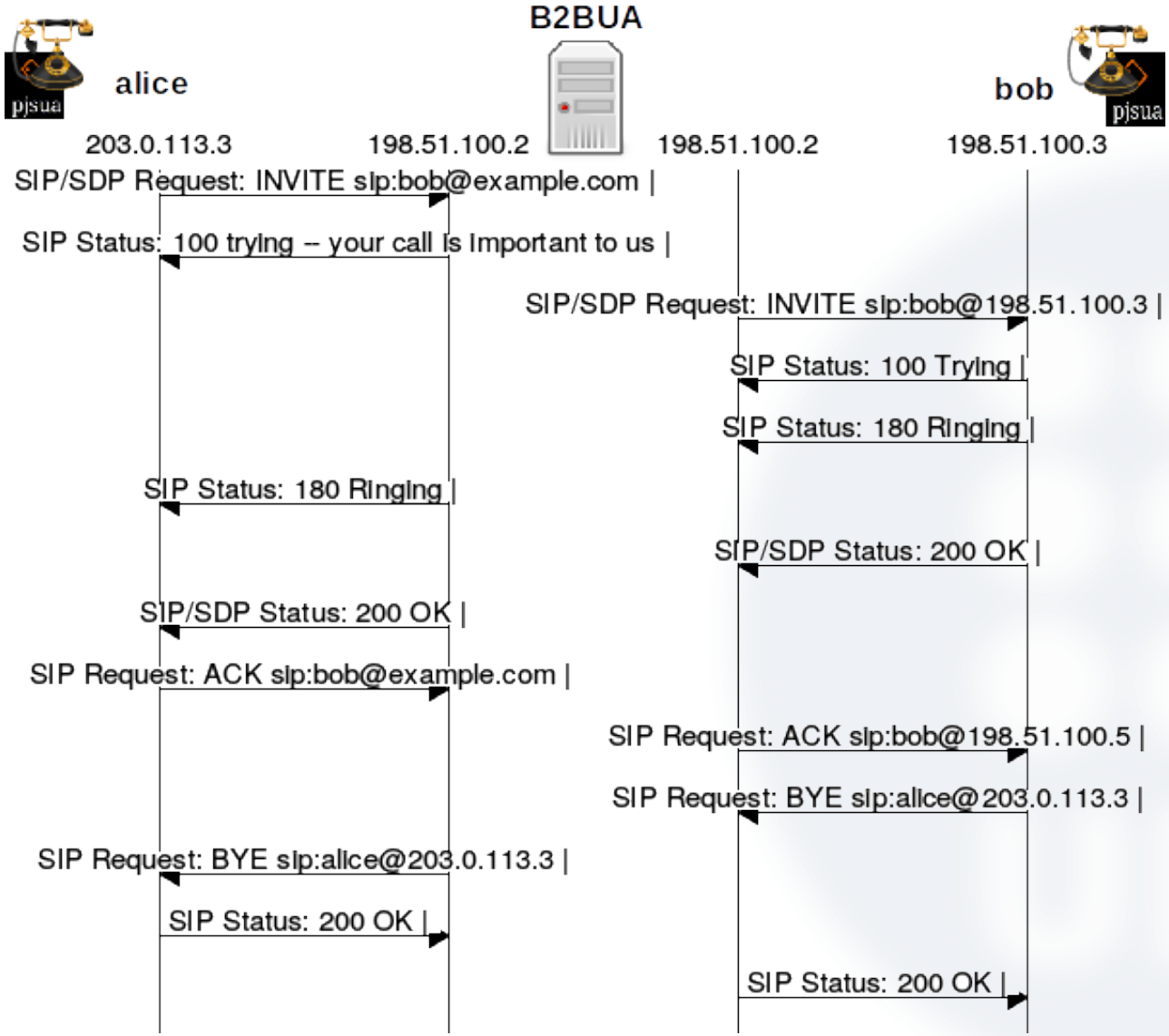
User Agents

- The User Agent (UA) manages the **SIP signaling** and comprises:
  - **UAC - User agent client**: logical entity that creates and sends requests.
  - **UAS - User Agent Server**: logical entity that generates responses.
  - During a session, a UA usually operates as both UAC and UAS.
- The UA also manages **multimedia streams**:
  - The UA should be able to describe multimedia streams.
  - Typically, SIP uses the Session Description Protocol (SDP).

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

B2BUA

A B2BUA is a **concatenation** UAS and UAC:



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

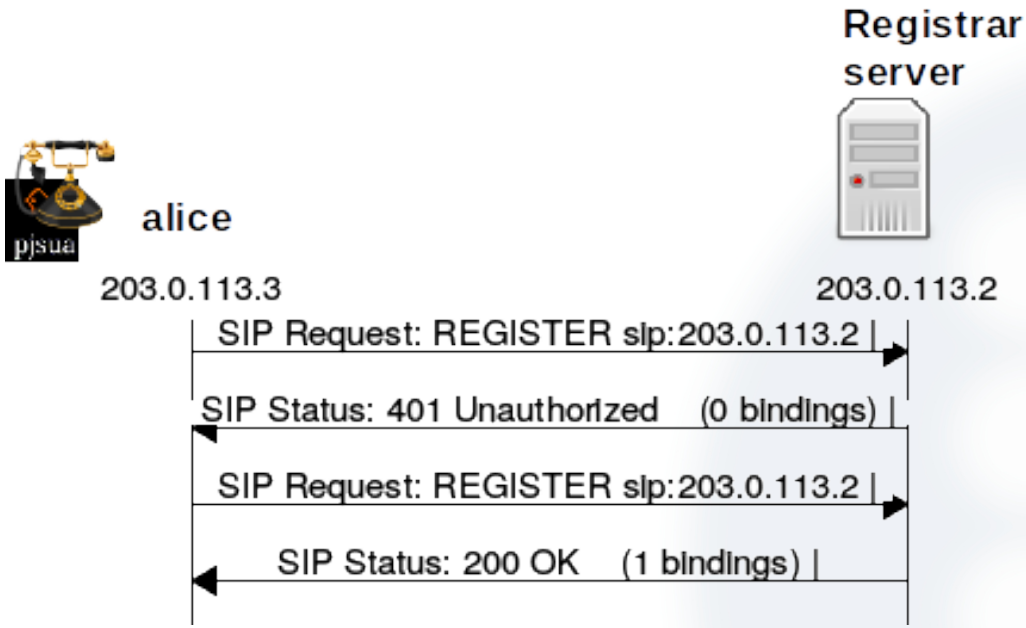
Registrar server I

- The registration process allows mobility to SIP users.
- This is not a mobility on established calls (call continuity).
- Each user has an **Address-of-Record (AOR)**:
  - An AOR is a SIP (or SIPS) URI pointing to a domain where there is a location service.
  - The location service can map the URI of the AOR to another URI where the user may be available.
  - Typically, the location service uses data coming from the registration process between the UA and a Registrar server.
  - The AOR can be seen as the ”public address“ of the user.

Registrar server II

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

- **Registration Process:**



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Registrar server III

- The UA sends a request with the REGISTER method:
  - This request goes to the Registrar server (in the **request URI**).
  - This method reports which contact address (in the **`Contact`** header) should be associated to the AOR (in the **`To`** header).

```
REGISTER sip:203.0.113.2 SIP/2.0
Via: SIP/2.0/UDP 203.0.113.3:5060;rport;branch=
z9hG4bKPjPUuQdSLFvSs62wtvszZq5QtM3yHhaYNK
Route: <sip:203.0.113.2;lr>
Max-Forwards: 70
From: <sip:alice@example.org>;tag=OeY3VoysTt6wcFhDEhXt9ghk7tfJah4r
To: <sip:alice@example.org>
Call-ID: b472q0.De5q0f4eTP2JXo2Jcku.1hNDp
CSeq: 26552 REGISTER
User-Agent: PJSUA v2.3 Linux-3.3.8/i686/glibc-2.11
Contact: <sip:alice@203.0.113.3:5060;ob>
Expires: 0
Content-Length:  0
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Registrar server IV

- If the Registrar server has been configured with **authentication**, it sends a response `401 Unauthorized`:

```
SIP/2.0 401 Unauthorized
Via: SIP/2.0/UDP 203.0.113.3:5060;rport=5060; branch=
z9hG4bKPjPUuQdSLFvSs62wtvszZq5QtM3yHhaYNK;received=203.0.113.3
From: <sip:alice@example.org>;tag=OeY3VoysTt6wcFhDEhXt9ghk7tfJah4r
To: <sip:alice@example.org>;tag=b27e1a1d33761e85846fc98f5f3a7e58.403e
Call-ID: b472q0.De5q0f4eTP2JXo2Jcku.1hNDp
CSeq: 26552 REGISTER
WWW-Authenticate: Digest realm="example.org",
nonce="VhJxxlYScJrjisKts5M7fWpHfkKFxoqS"
Server: kamailio (4.2.6 (i386/linux))
Content-Length: 0
```

- In this case authentication is required (**`WWW-Authenticate`** header).
- For this authentication, the UA and the Registrar share a secret (password).
- The Registrar sends a challenge (nonce), which will be used to demonstrate that the UA knows the password.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Registrar server V

- The UA must perform the following operations to send the response to the authentication challenge:

$$HA1 = MD5(username : realm : password)$$

$$HA2 = MD5(method : digestOfRequestURI)$$

$$response = MD5(HA1 : nonce : HA2)$$

- The UA resends the REGISTER request with the data required for authentication:

```
REGISTER sip:203.0.113.2 SIP/2.0
Via: SIP/2.0/UDP 203.0.113.3:5060;rport;branch=
z9hG4bKPjqwLrEeKmVZc0G1UXtjabLcPp9Ff4Ib06
Route: <sip:203.0.113.2;lr>
Max-Forwards: 70
From: <sip:alice@example.org>;tag=OeY3VoysTt6wcFhDEhXt9ghk7tfJah4r
To: <sip:alice@example.org>
Call-ID: b472q0.De5q0f4eTP2JXo2Jcku.1hNDp
CSeq: 26553 REGISTER
User-Agent: PJSUA v2.3 Linux-3.3.8/i686/glibc-2.11
Contact: <sip:alice@203.0.113.3:5060;ob>
Expires: 0
Authorization: Digest username="alice", realm="example.org",
nonce="VhJxxlYScJrjisKts5M7fWpHfkKFxoqS", uri="sip:203.0.113.2",
response="e3dcbb062fe3e9b53357de3aa741864e"
Content-Length:  0
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Registrar server VI

- If the REGISTER is correct, Registrar performs the binding and sends a `200 OK` response:

```
SIP/2.0 200 OK
Via: SIP/2.0/UDP 203.0.113.3:5060;rport=5060;branch=
z9hG4bKPjqwLrEeKmVZc0G1UXtjabLcPp9Ff4Ib06;
received=203.0.113.3
From: <sip:alice@example.org>;tag=OeY3VoysTt6wcFhDEhXt9ghk7tfJah4r
To: <sip:alice@example.org>;tag=b27e1a1d33761e85846fc98f5f3a7e58.021a
Call-ID: b472q0.De5q0f4eTP2JXo2Jcku.1hNDp
CSeq: 26553 REGISTER
Server: kamailio (4.2.6 (i386/linux))
Content-Length: 0
```

- In this case, a binding without expiration was performed:
  - The client sent the **`Expires: 0`** header.
  - In general, bindings may have a duration, where the customer proposes an expiry date, and the Registrar assigns it.
  - Temporary bindings expire if not renewed.
  - A binding can also be terminated by a deregistration.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP Proxies I

- A proxy server is an intermediary entity that routes requests on behalf of clients (UAs).
- Proxy servers are mainly in charge of the **SIP routing**:
  - They ensure that requests are sent to another entity nearer the destination UA.
  - A proxy interprets and, if necessary, rewrites specific parts of requests before forwarding.
  - Generally, they do not initiate SIP requests, just forward them.
  - Exceptions are the generation of ACK and CANCEL requests by proxies in certain cases (for instance, forking proxy).
  - They can answer requests of UAs.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP Proxies II

- Proxy servers do not have multimedia capabilities.
- Proxies are also useful for implementing policies at SIP level:
  - For example, before forwarding a request, a proxy can ensure that the corresponding user is allowed to establish the session.
- Types of proxies:
  - According to the information stored, they can be classified into: **stateless**, **transaction stateful** and **call stateful**.
  - Depending on if they handle incoming or outgoing requests to an administrative domain, they can be classified in: **inbound** and **outbound**.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Stateless Proxies

- A stateless proxy processes each message, but without considering the state (context).
- For example, a stateless proxy could not correlate a request with its corresponding response.
- They perform simple forwarding of messages.
- As they do not deal with state, they are highly scalable.
- Their functionality is limited: basically they are used to distribute load.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Stateful Proxies

- Stateful proxies are required to implement any advanced functionality, such as:
  - Forward on busy.
  - Voice mail.
  - Call Accounting, etc.
- There are two types:
  - **Transaction stateful**.
  - **Call stateful**.

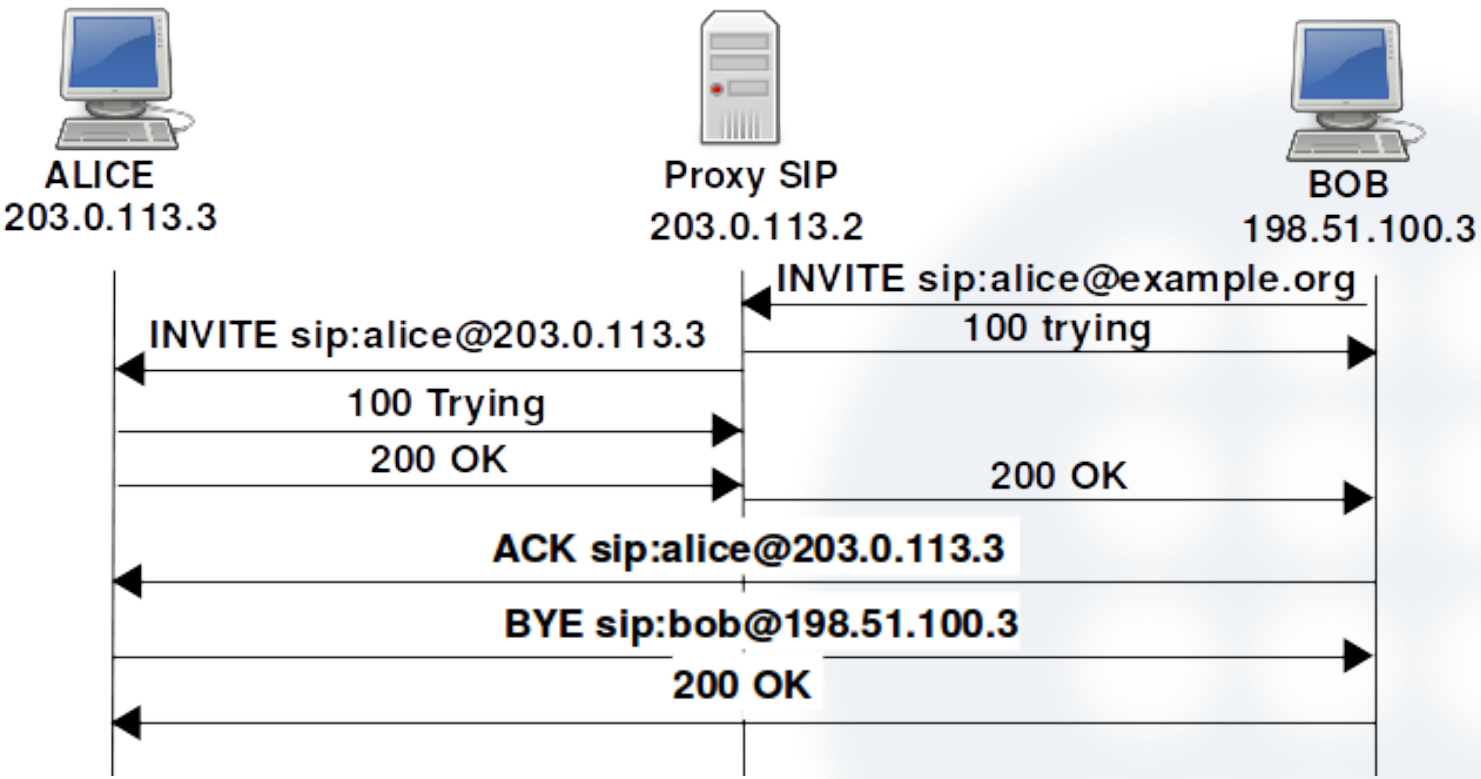
Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Transaction Stateful I

- Information about each SIP transaction is held in the memory of the proxy until the transaction is completed.
- These proxies can recognize requests, errors or retransmissions.
- They do not maintain information about the dialogue (recall a SIP dialog may consist of several transactions).
- As a a consequence, a transaction stateful proxy does not know if a call is in progress.
- It cannot calculate the length of a call in real time (call accounting).
- Therefore, it cannot end an ongoing call based on its duration.
- In the following example, the proxy **is transaction stateful**.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Transaction Stateful II



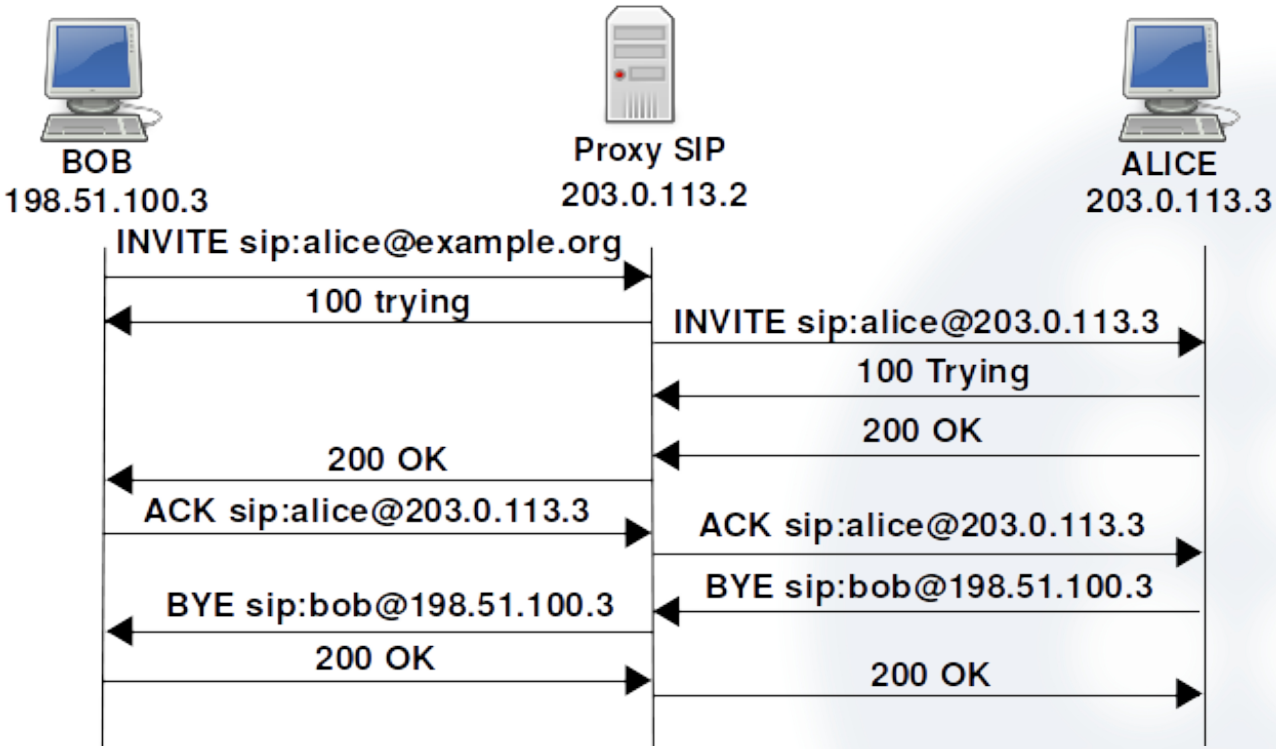
Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Call Stateful I

- They maintain information of the all SIP dialog (call).
- They can calculate durations and end calls in real-time (call accounting).
- They are the least scalable of all proxies, but they offer more functionality.
- They need to be present in the signaling path during all the call.
- The **`record-route`** header is the one that forces that all transactions within a dialogue are routed through the call stateful proxy.
- In the following example, the proxy **can be** call stateful.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

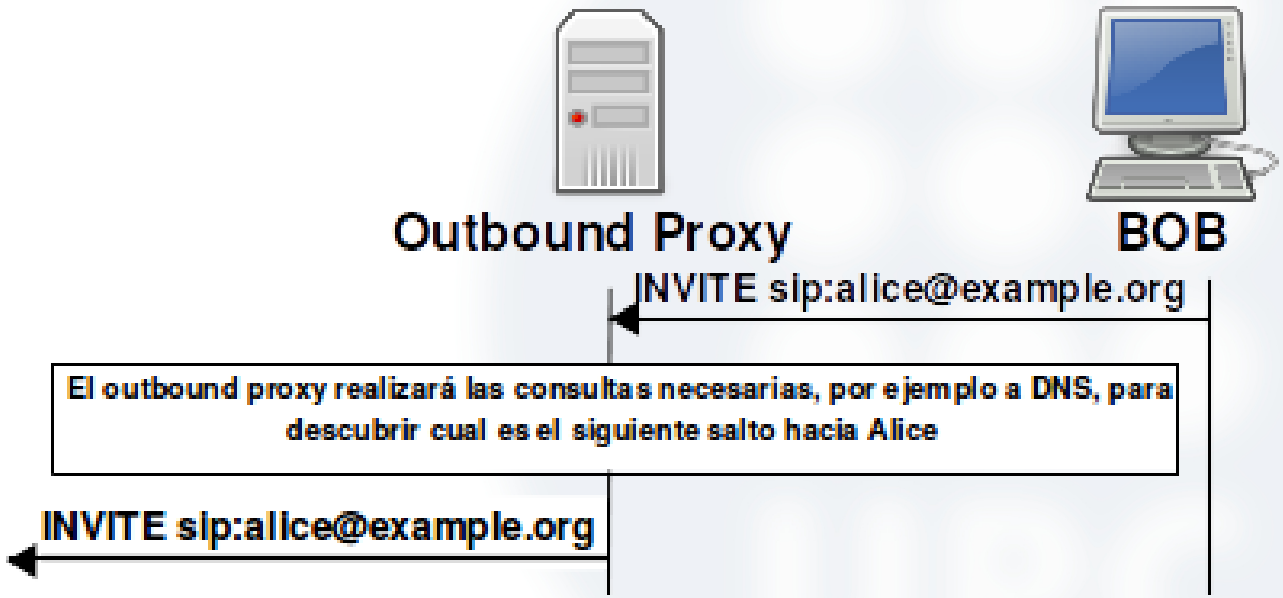
Call Stateful II



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outbound Proxy

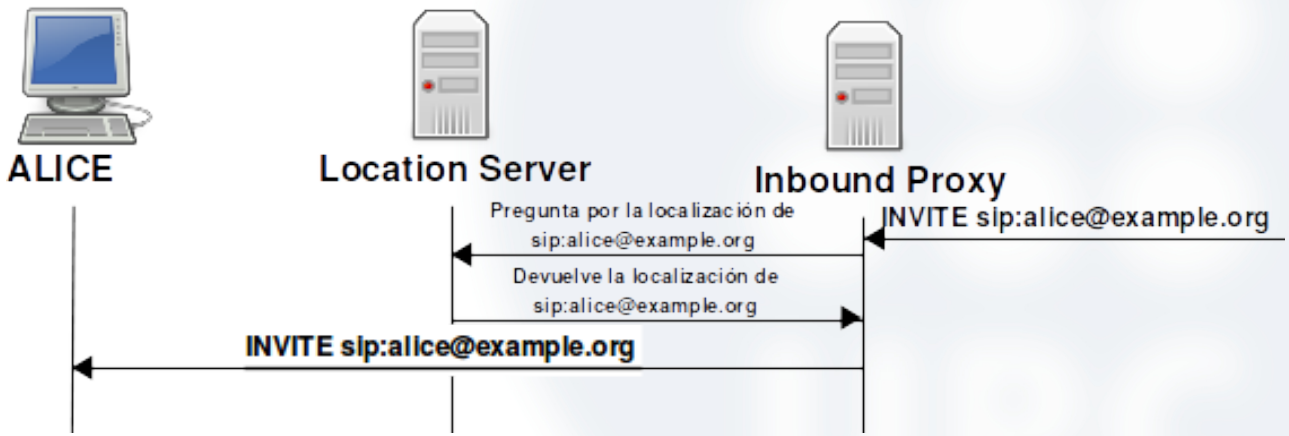
- An outbound proxy helps the UAs to route their outgoing requests.
- UAs are generally configured to route all SIP requests through the same outbound proxy.



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Inbound Proxy

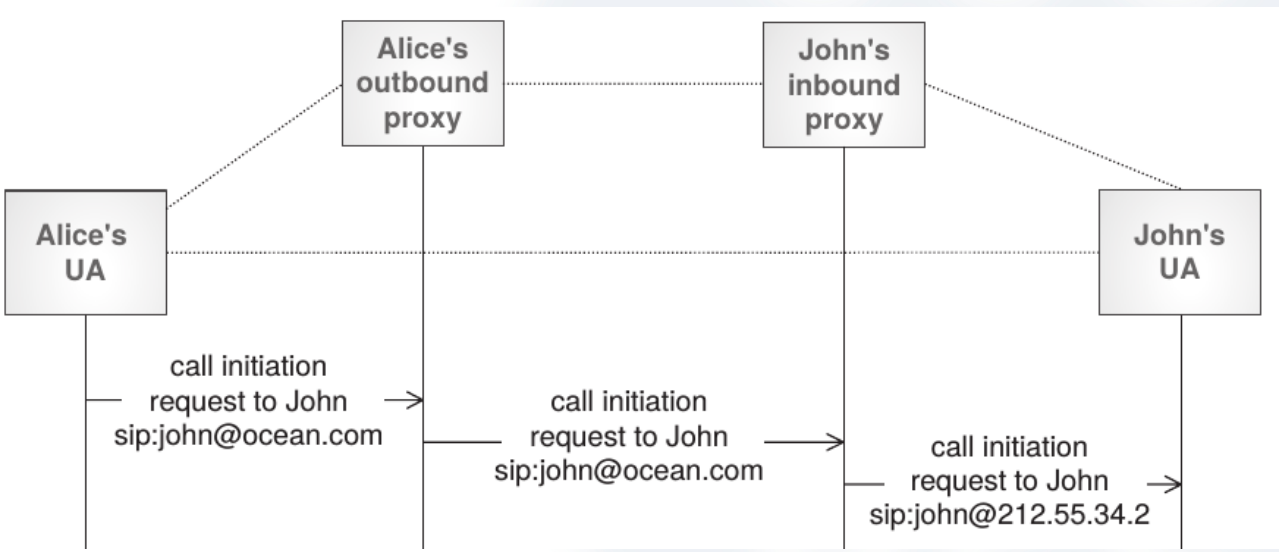
- An inbound proxy SIP handles incoming requests to a domain.
- It routes incoming requests to the UA.
- It checks the location service (in the Registrar server) to determine the contact URI of the UA, and then forwards requests to that address.



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SIP trapezoid

- A very common scenario is the one in which requests to start a call have to pass through both an outbound and inbound proxy.
- This scenario is commonly known as the ”SIP Trapezoid“.
- It is also very common to have the inbound, outbound and Registrar all integrated into a single physical computer.



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Outline

1 Sessions

2 SDP

3 SIP
- Introduction
- SIP basics
- Headers
- SIP Architecture
- DDDS & ENUM

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Dynamic discovery I

- One of the most important functions of SIP is that it allows **a user to register at different locations**.
- To enable this SIP scenario:
  - The called party must be in a domain that has an inbound proxy and a Registrar (may be in the same server).
  - The caller must have the AoR of the called party. Example: *sip:alice@example.com*.
- Although the above conditions are met, there are still **important issues to solve ...**

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Dynamic discovery II

1. *What is the IP address to send requests for this user (e.j INVITE)?*
2. *What transport protocol or protocols can be used for SIP (UDP, TCP, SCTP, TLS, DTLS, SCTP over DTLS)?*
3. *What are the ports to send SIP requests?*

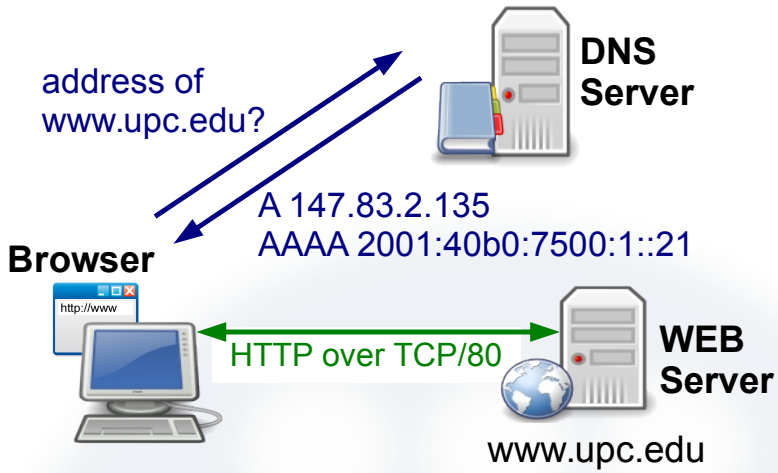
- Even, non SIP URIs may be used: alice@example.com.
- In this case, you might also ask:

4. *In this address, is there a user that supports SIP?*

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Basic resolution of URIs

- To solve the above questions, have a look at how URIs are resolved on the Internet.
- Example: WEB service.



- For example, to resolve the URI http://www.upc.edu:
  - A request is made to the DNS service (DNS lookup) using the FQDN (Fully Qualified Domain Name) www.upc.edu.
  - The DNS sends a response with the corresponding address (or addresses).
  - The DNS may return one or more records of type A (Address, IPv4) and AAAA (IPv6) with possible directions.
  - The client selects one of the addresses.
  - The transport protocol and the port used for http is by default TCP/80.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

SRV Records

SRV Record

- RFC2782 defines a new type of DNS record for providing greater flexibility to the process of resolving URIs.
- For each service and transport protocol, a SRV record [a] (SeRVice) defines how to assign a server to this service (including a load sharing scheme).

[a] *SRV records are not used in "legacy" services like web or e-mail.*

```
_service._protocol.name TTL class SRV priority weight port destination
```

- **priority** indicates the priority of the server. The client should contact first the server with lower priority.
- **weight** is used to select a server among the ones with the same priority. A larger weight means that the candidate must be more likely selected.
- **port** used by the transport protocol.
- **destination** is the FQDN of the server that provides the service (this FQDN should have A or AAAA records associated).

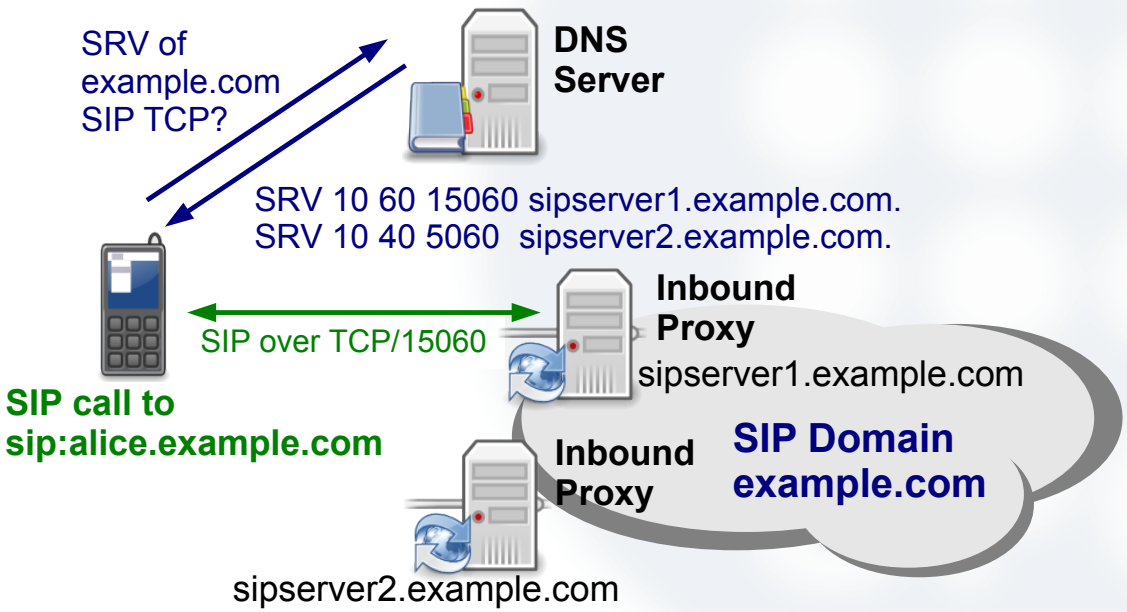
Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Resolution of URIs with SRVs

- Examples of SRV records:

```
_sip._tcp.example.com. 86400 IN SRV 10 60 15060 sipserver1.example.com.
_sip._tcp.example.com. 86400 IN SRV 10 40 5060  sipserver2.example.com.
```

- The service, transport protocol and domain must be provided to obtain the record.
- A and AAAA records should also be consulted to get the addresses of the servers in the SRV records.



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

DDDS

- To use a SRV record **you must know the transport protocol of the service.**
- RFC3263 states that SIP can use DNS:

cite from RFC3263

The Session Initiation Protocol (SIP) uses DNS procedures to allow a client to resolve a SIP Uniform Resource Identifier (URI) into the IP address, port, and transport protocol of the next hop to contact. It also uses DNS to allow a server to send a response to a backup client if the primary client has failed.

- To provide an even more flexible mechanism, RFC3403 defines a **Dynamic Delegation Discovery System (DDDS)**.
- DDDS uses a newly defined type of DNS record, known as **Name Authority Pointer or NAPTR**.

NAPTR records

- When a client queries the DNS by using NAPTR and giving the domain name, the result (if everything is properly configured) is one or more NAPTR records.
- A NAPTR record can provide:
  1. FQDN name to obtain the addresses of servers (A/AAAA records).
  2. A name of a SRV record to know a load sharing scheme per service and transport protocol.
  3. A new URI to consult again.
  4. A new URI modified with a regular expression to consult again.
- Format:

```
owner-name ttl class NAPTR order preference flag params regexp replace
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

NAPTR & SIP I

- RFC3263 recommends to use a **NAPTR lookup to the DNS using the basic domain** when a user is initially contacted.
- Example when contacting sip:alice@example.com:
  1. Start a NAPTR lookup to example.com.
  2. Only if this lookup fails, try lookups to SRV records.
- The reason is that:
  - NAPTR allows administrators to define preference in services to be used.
  - SRVs only provide operational details for a service (hostnames, ports and load sharing).

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

NAPTR & SIP II

```
owner-name ttl class NAPTR order preference flag params regexp replace
```

- The following NAPTR records could be used to define access to SIP using these inbound proxies of domain example.com:

```
example.com. IN NAPTR 10 10 "s" "SIPS+D2T" "" _sips._tcp.example.com.
example.com. IN NAPTR 20 10 "s" "SIP+D2T"  ""  _sip._tcp.example.com.
example.com. IN NAPTR 30 10 "s" "SIP+D2U"  ""  _sip._udp.example.com.
```

- Essentially, previous records indicate that:
  - If the user supports SIP over TLS, it should use this service first (order 10).
  - In second place, SIP over TCP (order 20).
  - In third place, SIP over UDP (order 30).
  - The parameters to use SIP for each transport protocol are defined in the SRV records.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

NAPTR & SIP III

```
owner-name class NAPTR order pref flag params regexp replace
example.com. IN  NAPTR  10    10   "s"  "SIPS+D2T" ""  _sips._tcp.example.com.
example.com. IN  NAPTR  20    10   "s"  "SIP+D2T"  ""  _sip._tcp.example.com.
example.com. IN  NAPTR  30    10   "s"  "SIP+D2U"  ""  _sip._udp.example.com.
```

- **order** (16 bits) specifies the order in which NAPTR records must be processed (lower values higher priority).
- **preference** (16 bits) used to give priority to NAPTR records with the same `order` value (lower values higher priority).
  - When a client finds a NAPTR record with an acceptable service for its request, it must not analyze other records with different `order`.
  - The `preference` parameter allows administrators to redirect clients to the more capable servers or to the most efficient transport protocols.
  - A client may prefer a NAPTR with lesser `preference` priority because it does not support a certain protocol.
  - Note: The load sharing is implemented with SRVs.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

NAPTR & SIP IV

```
owner-name class NAPTR order pref flag params regexp replace
example.com. IN  NAPTR  10    10   "s"  "SIPS+D2T" ""  _sips._tcp.example.com.
example.com. IN  NAPTR  20    10   "s"  "SIP+D2T"  ""  _sip._tcp.example.com.
example.com. IN  NAPTR  30    10   "s"  "SIP+D2U"  ""  _sip._udp.example.com.
```

- **flag** is specific to each application. In this case, the value "*s*" means that the field **replace** contains the name of a SRV.
- **params** defines the type of application and transportation. The format is *protocol+rs*.
  - *protocol*: protocol used by the DDDS application.
  - *rs* (*resolution service*): 0 or more *rs* separated by *+*. It indicates the transport layer protocol.
  - Examples for SIP:

```
"SIP+D2T": SIP over TCP
"SIP+D2U": SIP over UDP
"SIPS+D2T": SIP with TLS over TCP
"SIP+D2S": SIP over SCTP
```

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

Example NAPTR & SIP

```
example.com IN SOA ns.example.com. root.example.com. (
          2003032001
          10800
          3600
          604800
          86400 )

example.com.              43200 IN NS      ns.example.com.
;
ns.example.com.           43200 IN A       10.0.0.20
sipserver1.example.com.   43200 IN A       10.0.0.21
sipserver2.example.com.   43200 IN A       10.0.0.22
;
_sip._udp.example.com.    43200 IN SRV 0 0 5060  sipserver1.example.com.
_sip._udp.example.com.    43200 IN SRV 1 0 5060  sipserver2.example.com.
_sip._tcp.example.com.    43200 IN SRV 0 4 5060  sipserver1.example.com.
_sip._tcp.example.com.    43200 IN SRV 0 2 5060  sipserver2.example.com.
_sips._tcp.example.com.   43200 IN SRV 0 0 5060  sipserver1.example.com.
_sips._tcp.example.com.   43200 IN SRV 0 0 5060  sipserver2.example.com.
;
example.com. IN NAPTR 0 0 "s" "SIPS+D2T" "" _sips._tcp.example.com.
example.com. IN NAPTR 1 0 "s" "SIP+D2T"  "" _sip._tcp.example.com.
example.com. IN NAPTR 2 0 "s" "SIP+D2U"  "" _sip._udp.example.com.
```

NAPTR: flag, regexp & replace

```
owner-name class NAPTR order pref flag params regexp replace
example.com. IN  NAPTR  10    10   "s"  "SIPS+D2T" ""  _sips._tcp.example.com.
example.com. IN  NAPTR  20    10   "s"  "SIP+D2T"  ""  _sip._tcp.example.com.
example.com. IN  NAPTR  30    10   "s"  "SIP+D2U"  ""  _sip._udp.example.com.
```

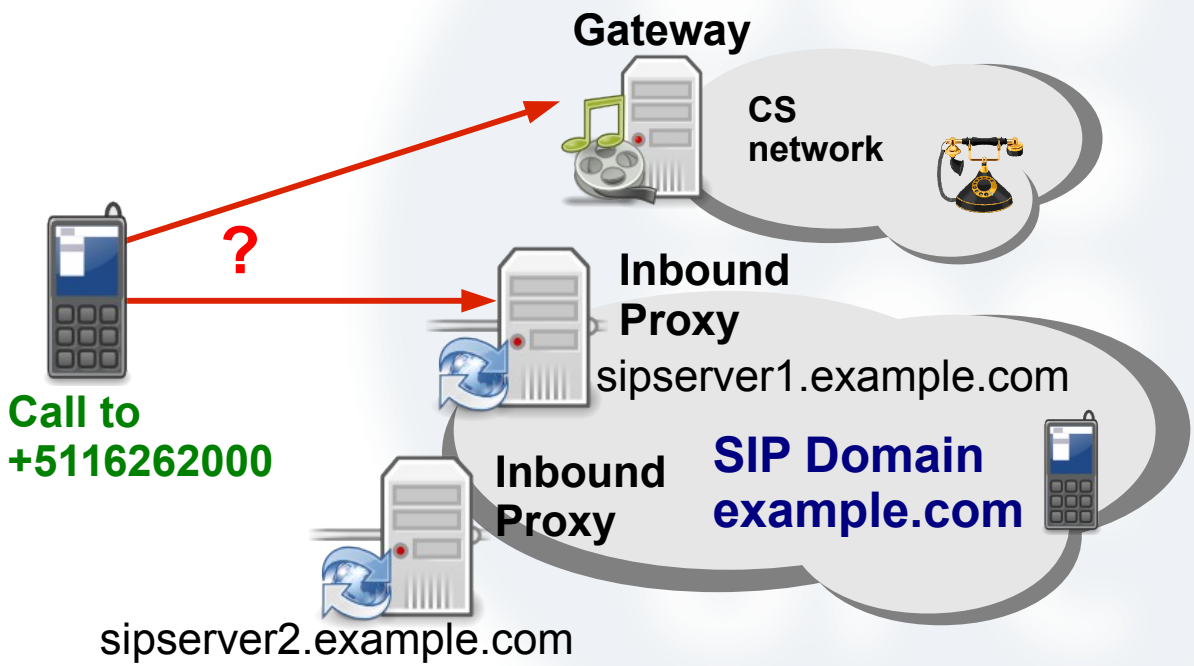
- **flag** depends on the application, and it can take several values:
  - **u**: In **regexp** there is a valid URI.
  - **s**: In **replace** there is the FQDN of a SRV register.
  - **a**: In **replace** there is the FQDN of a A (or AAAA) register.
  - “ ” (empty string): used by ENUM.
- **regexp** is presented in quotation marks (“ ”).
  - It contains an expression of substitution
- **replace** is a string containing the FQDN to consult depending on the contents of *flag*.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

E.164 & ENUM I

- One of the open questions is how to manage a call that goes from a SIP domain to a user in the PSTN network:
  - In this case, the user uses telephone numbers to identify the call.
  - Telephone numbers are defined in the E.164 standard, following the format +34934011234 or tel:+34934011234.
  - Actually, the E.164 numbering may identify a user in the PSTN but also in the SIP network.

How to identify a server (proxy o gateway) to manage this call?



Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

E.164 & ENUM II

- The IETF ENUM work group specified a way to transform FQDNs to E.164 numbers, which can be consulted in the DNS.
- ENUM (E.164 Number to URI Mapping) reserve the suffix e164.arpa.
- The transformation is performed as follows:
  1. Removing the +
  2. The digits of the E.164 numbering are inverted and separated by dots.
  3. The suffix e164.arpa. is included.
- Example:

```
+34934011234 → 4.3.2.1.1.0.4.3.9.4.3.e164.arpa.
```

- Then, the FQDN obtained with ENUM is used to obtain NAPTR records that will allow to find an entity to handle the call.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

NAPTR for ENUM I

- ENUM defines how to use the **flag** field specifically:
  - Only *"u"* and " " are used.
  - The flag *u* indicates that the result of applying the *regexp* field will be a valid URI for the protocol defined in *params*, and that the *replace* field should be ignored.
  - The empty flag (" ") indicates that the *replace* field contains the FQDN for a new NAPTR search.
- The **params** field has the format E2U+enumservice (E2U means E.164 to URI),
- Common examples:
  - E2U+sip defines that in the URI of the NAPTR record appears a SIP AoR (see RFC3824).
  - E2U + mailto defines that in the URI of the NAPTR record appears an email address.

Sessions
SDP
SIP
Introduction
SIP basics
Headers
SIP Architecture
DDDS & ENUM

NAPTR for ENUM II

NAPTR Examples of ENUM:

1. +34934011234 becomes the SIP URI sip:info@upc.edu to be used in the first instance. If the SIP contact is not possible, the email info@upc.edu can be used:

```
$ORIGIN 1.0.4.3.9.4.3.e164.arpa.
4.3.2.1 IN NAPTR 10 10 "u" "E2U+sip"    "!^.*$!sip:info@upc.edu!" .
4.3.2.1 IN NAPTR 10 20 "u" "E2U+mailto" "!^.*$!mailto:info@upc.edu!" .
```

2. +34934011234 is translated to SIP URI sip:usr2000@upc.edu:

```
$ORIGIN 1.0.4.3.9.4.3.e164.arpa.
4.3.2.1 IN NAPTR 10 10 "u" "E2U+sip" "!^\\+34401(.+)$!sip:usr\\1@upc.edu!" .
```